



EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 6a_Corona Break



MIPS Addressing Modes

Addressing modes refer to the **ways to specify an operand or a memory address in an instruction.**

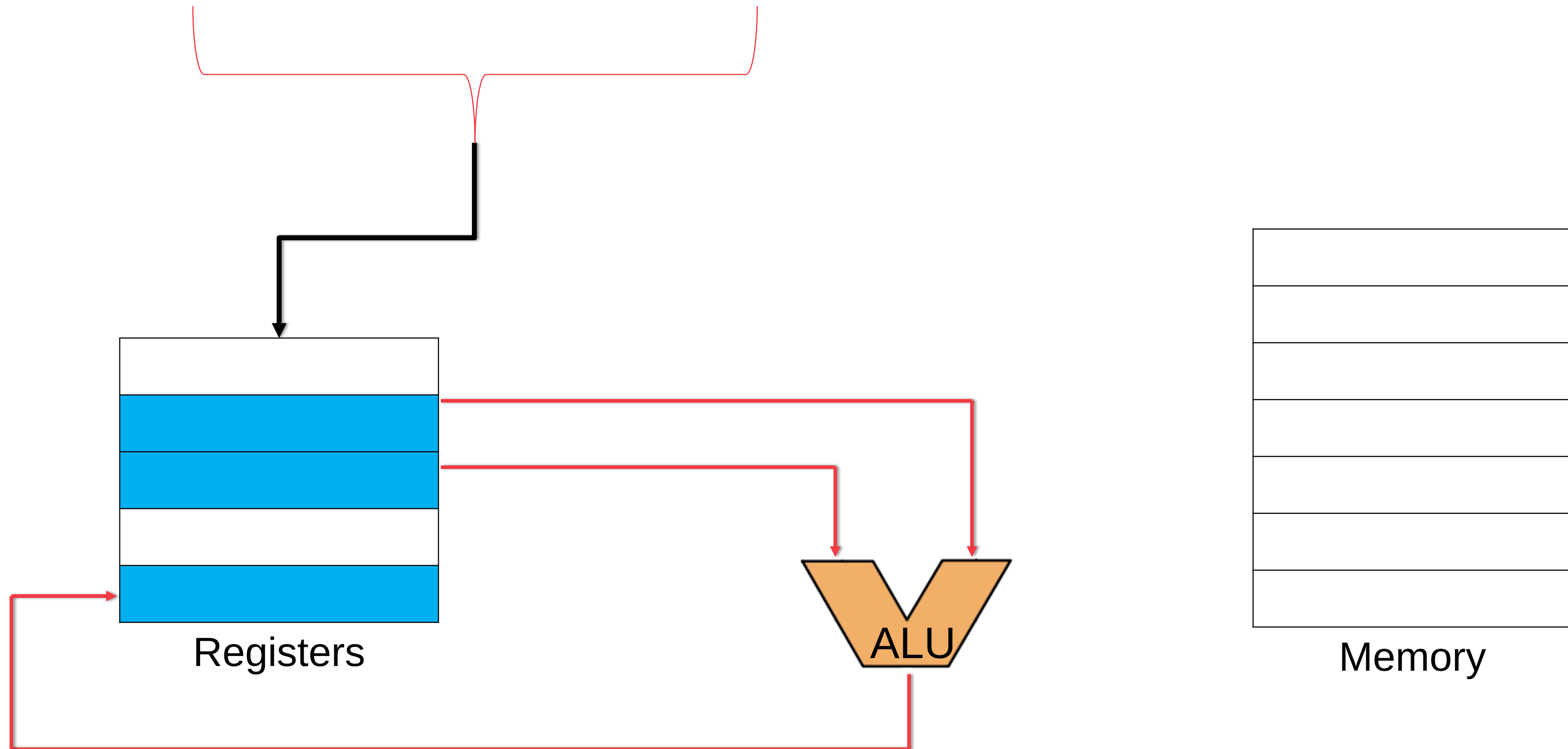
1. Register
2. Immediate
3. Base or Displacement
4. PC-relative
5. Pseudo-direct
6. Register-Indirect

Register Addressing Mode

- Operands are in registers
- Example:
 - add \$t0, \$S1, \$ S2
- It takes n bits to address 2^n registers

Op	rs	rt	rd	shamt	Funct
6	5	5	5	5	6
32 bits					

Register Addressing Mode



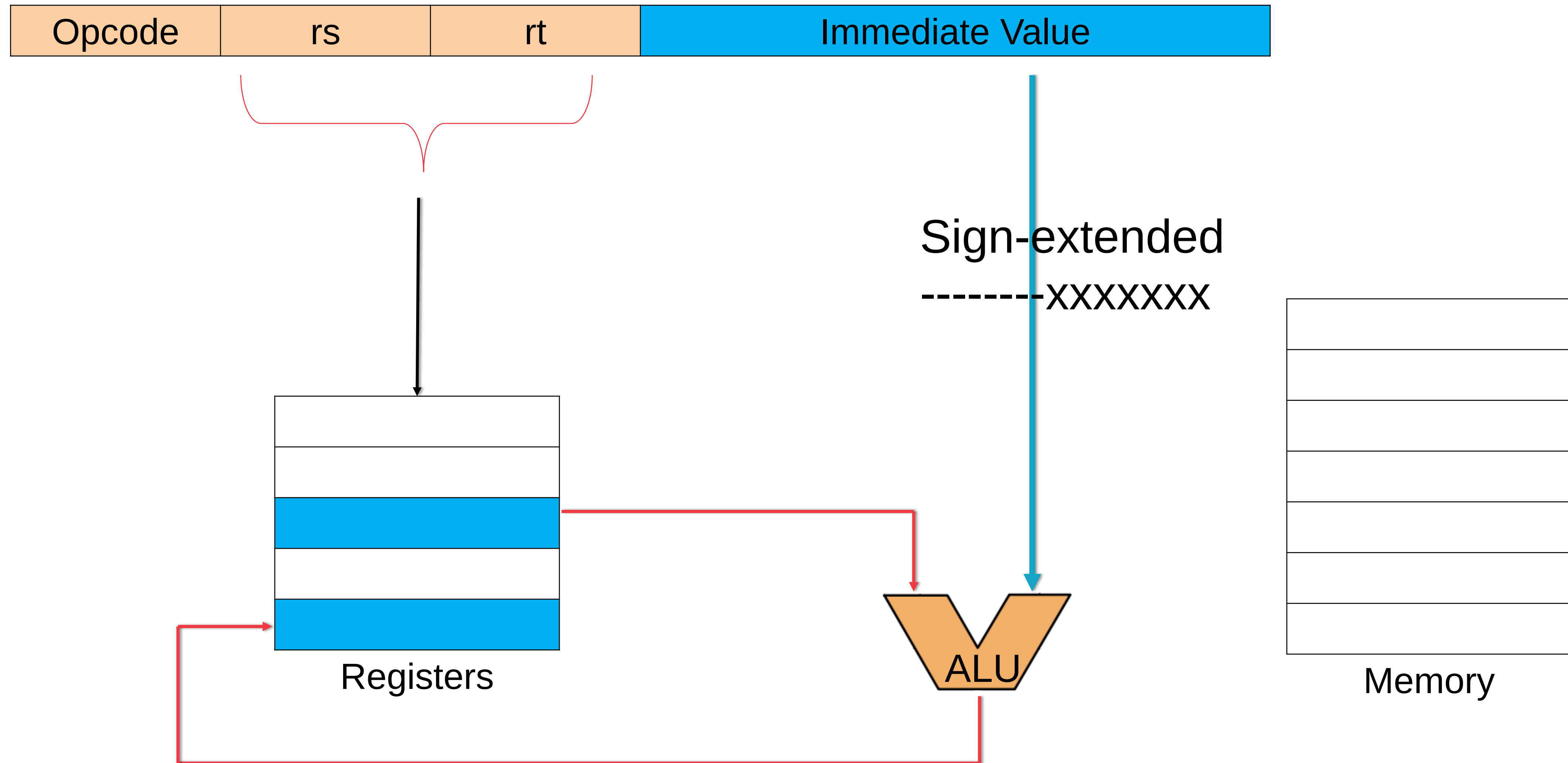
Immediate Addressing Mode

- The operand is embedded inside the encoded instruction
- Example:
 - `addi $t0, $S1, 4`
- 16-bit two's-complement number:
 - $-2^{15} \text{ (-32,768)} \leq \text{Value} \leq + 2^{15} - 1 \text{ (32,767)}$

Op	rs	rt	Immediate Value
6	5	5	16
32 bits			

Immediate Addressing Mode

Example: addi or similar



Base (or Base-offset or displacement) Addressing Mode

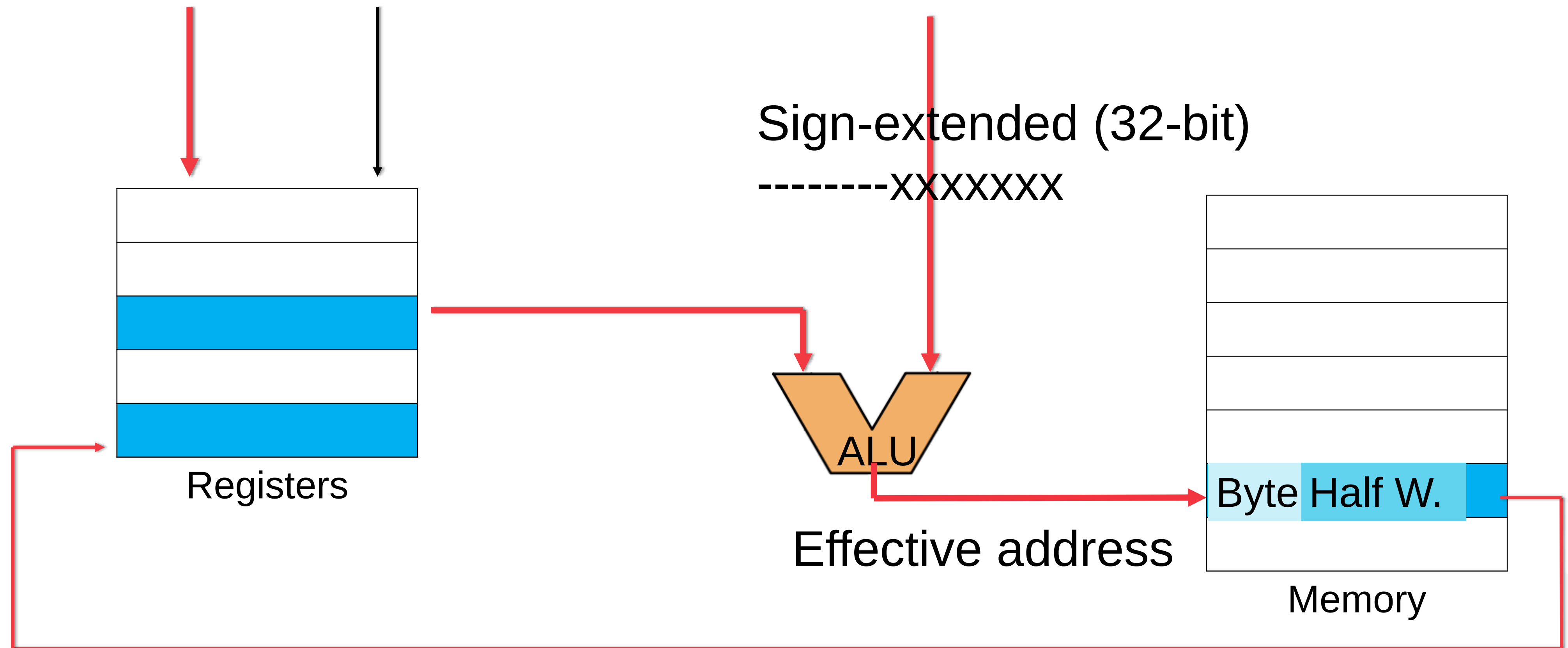
- The address of the operand is the sum of the immediate and the value in a register (rs)
- Example:
 - lw \$t0, 4(\$S1)
- 16-bit immediate is a two's-complement number

Op	rs	rt	Immediate Value
6	5	5	16
32 bits			

Base (or Base-offset or displacement) Addressing Mode

Example: lw \$t0, 4(\$S1)

Opcode	rs	rt	Immediate Value
			16 bits



PC-relative Addressing Mode

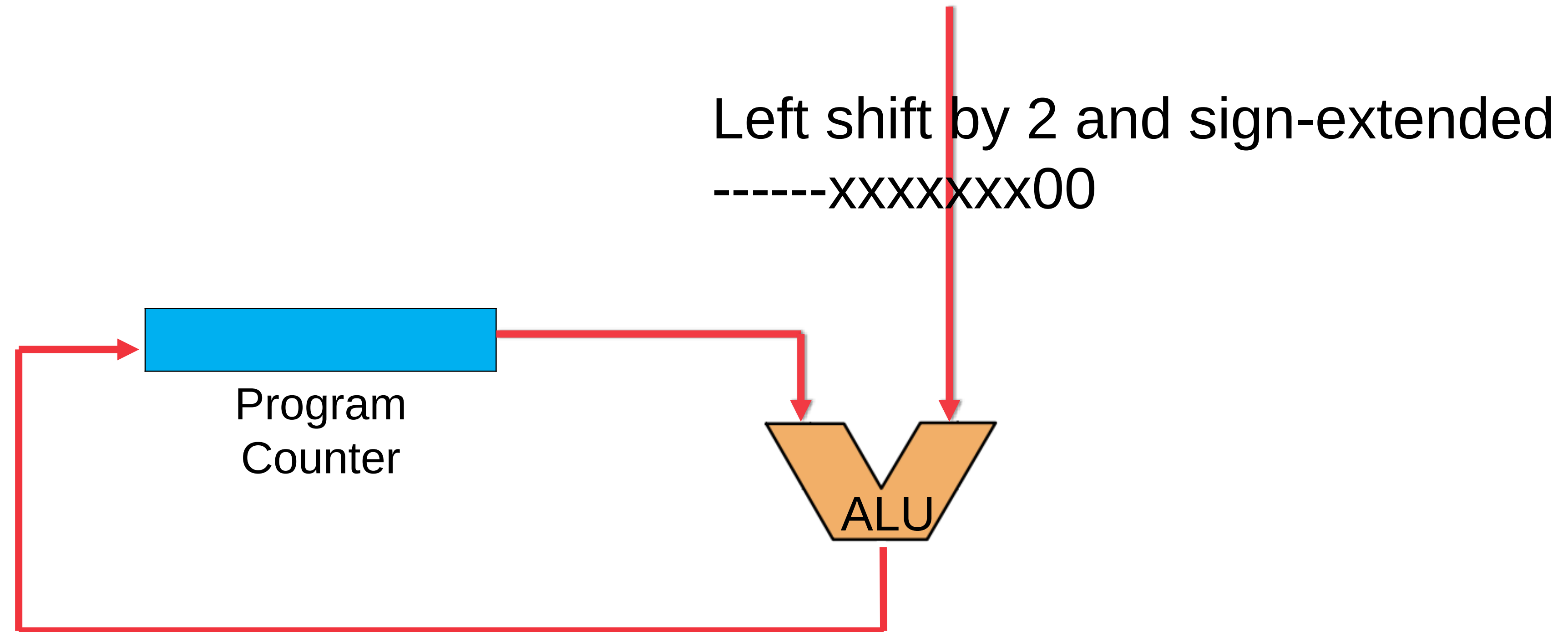
- The value in the immediate field is interpreted as an offset of the next instruction (PC + 4 of current instruction)
- Example:
 - `beq $t0, $t1, myLabel`
- 16-bit immediate is a two's-complement number

Op	rs	rt	Immediate Value
6	5	5	16
32 bits			

PC-relative Addressing Mode

Example: beq \$t0, \$t1, myLabel

Opcode	rs	rt	Immediate Value
			16 bits



Pseudo-direct (or Direct) Addressing Mode

- **Direct Addressing:** the address is “the immediate” i.e. encoded in the instruction. 32-bit address cannot be embedded in a 32-bit instruction (MIPS Inst. Size)
- **Pseudo-direct addressing:** 26 bits of the address is embedded as the immediate, and is used as the instruction offset within the current 256 MB (64 MWords) region, defined by the most significant 4 bits of the PC reg.
- **Example:**
 - `j myLabel`

Op	Pseudo-address
6	26
32 bits	

Pseudo-direct Addressing Mode

Example: j myLabel

Op		Pseudo-address	
31	26	25	0

Left shift by 2



Program Counter

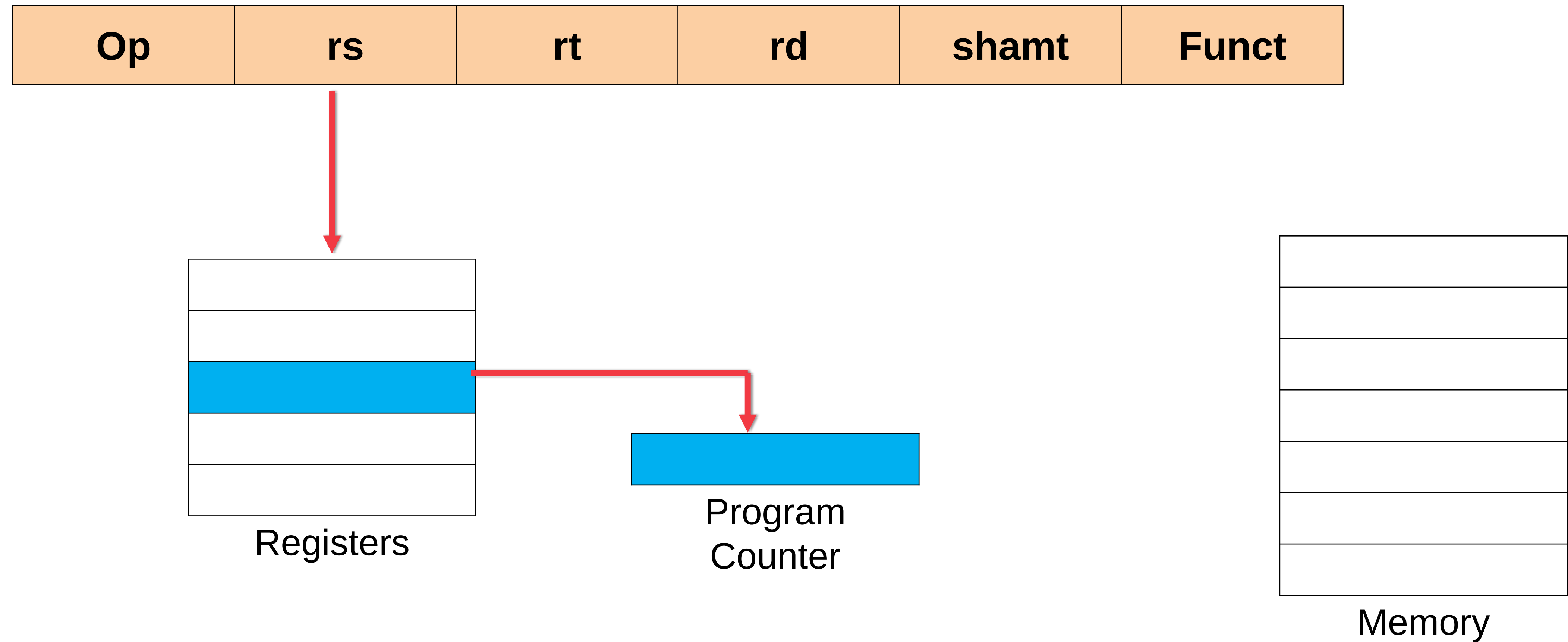
Register-Indirect Addressing Mode

- The effective address is placed inside the register.
- Special case of base addressing where offset is 0
- Example: (jr, jalr)
 - jr \$31 (**R-type Instruction**)

Op	rs	rt	rd	shamt	Funct
000000	rs	00000	00000	00000	001000

Register-Indirect Addressing Mode

Example: jr \$s0



Caution: Addressing mode is not Instruction type

- **Addressing mode** is how an address (memory or register) is determined
- **Instruction type** is how the instruction is put together – encoding.
- **Examples:** addi, beq/bne, lw/sw are all I-type instructions.
 - addi uses immediate addressing mode (and register)
 - beq uses pc-relative addressing (and register)
 - lw uses base addressing (and register)

Summary- MIPS Addressing Modes

- **Register Addressing:** a source or destination operand is specified as one of the registers \$0-31.
- **Immediate Addressing:** where the operand is a constant within the instruction itself
- **Base-displacement Addressing :** a data or instr. Memory location is specified as a signed offset from a register.
- **PC-relative Addressing :** a data or instr. Memory location is specified as an offset relative to the incremented PC.
- **Pseudo-direct Addressing :** the memory address is (mostly) embedded in the instruction.
- **Register-Indirect Addressing :** the value of the effective address is placed inside the register.

MIPS PC-relative and Pseudo-direct Addressing Example

```
Loop: sll $t1,$s3,2      # Temp reg $t1 = 4 * i
      add $t1,$t1,$s6     # $t1 = address of save[i]
      lw  $t0,0($t1)      # Temp reg $t0 = save[i]
      bne $t0,$s5, Exit   # go to Exit if save[i] ≠ k
      addi $s3,$s3,1      # i = i + 1
      j   Loop           # go to Loop

Exit:
```

80000	0	0	19	9	2	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	2		
80016	8	19	19	1		
80020	2	20000				
80024	...					

Study Assignment

- **x86 addressing modes** from online resources
- **Article 2.19** Fallacies and Pitfalls, **pp. 159-160**
- **Article 2.20** Concluding Remarks, **pp. 161-163**